

# Presentacion Práctica 1A

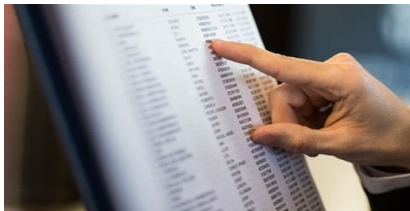
## Sistemas Informaticos 2

R. Marabini y Daniel Hernández Lobato

EPS-UAM

- Descripción de la aplicación
- Desacoplar servidor y cliente (API REST)
- Publicar código en las máquinas virtuales

# Descripción de la aplicación: Pago mediante Tarjeta

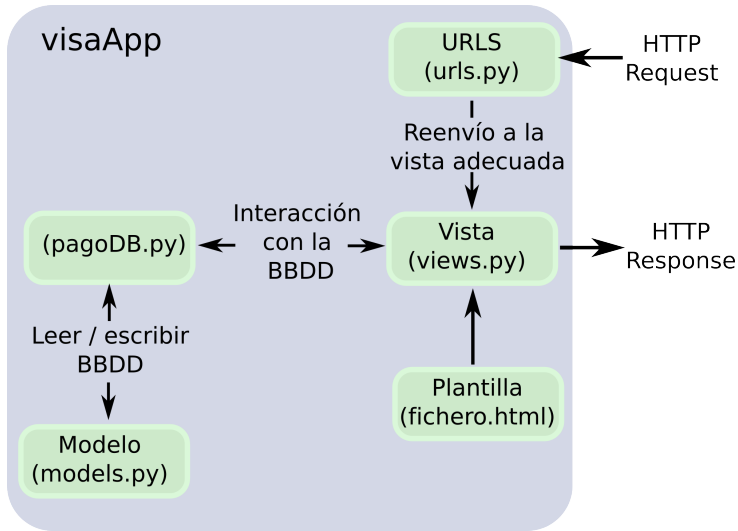


- (a) Existe una tabla de tarjetas (modelo “Tarjeta” en `models.py`)
- (b) Mediante el formulario “TarjetaForm” y la vista “`aportarinfo_tarjeta`” se verifican los datos

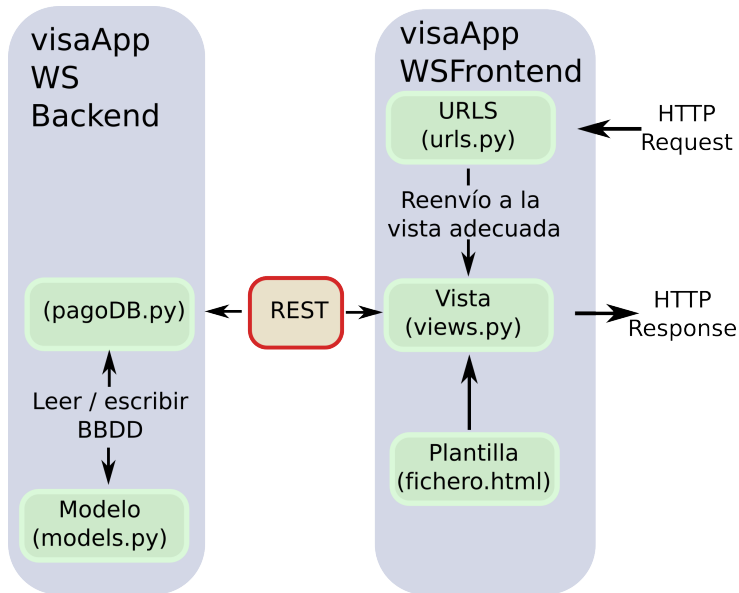


- (c) Se paga (PagoForm-`aportarinfo_pago`)
- (d) `utils (list-getpagos-/delete-delpagos-/create-testdb-)`

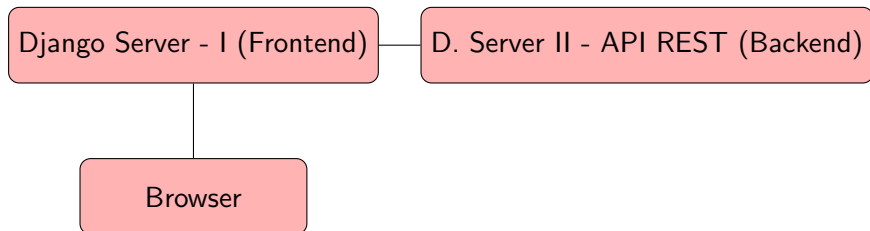
# Desacoplar Backend y Frontend - I (P1-base)



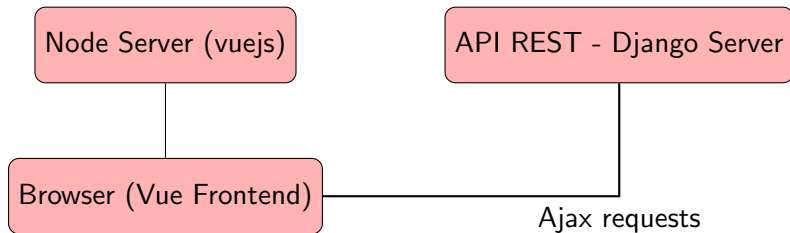
## Desacoplar Backend y Frontend - II



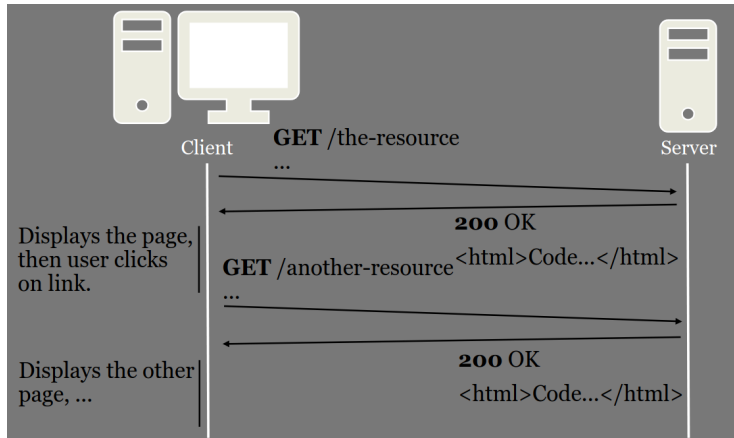
# Backend-Frontend (SI2)



# Backend-Frontend (PSI)



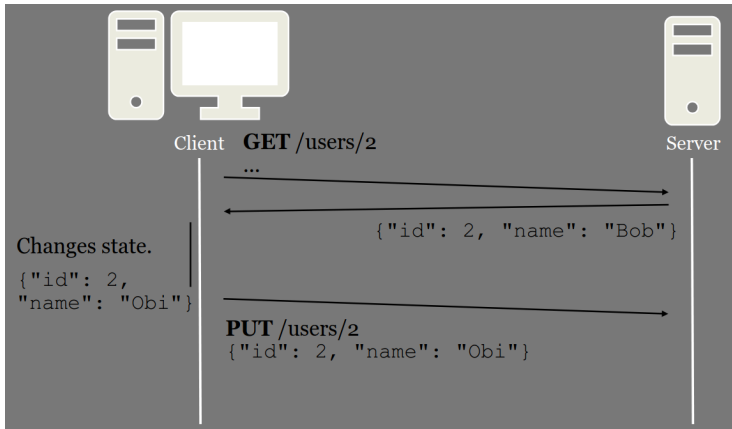
# API REST: Aplicación web tradicional



- El cliente debe entender HTML.
- Las páginas se repintan en cada llamada.
- El mismo código se envía múltiple veces (p.e. ficheros de estilo).



# API REST



# Convenciones seguidas por el API REST

- Usa URLs para identificar los recursos (`http://host/pago`)
- Usa métodos HTTP para especificar la operación
  - ▶ POST (crear)
  - ▶ GET (operaciones de solo lectura como listados)
  - ▶ PUT (actualizar)
  - ▶ DELETE (borrar)
- Usa el cabecero HTTP para especificar el formato de intercambio de datos(p.e. JSON)
- Usa los códigos HTTP para indicar el éxito o fracaso de la petición (200 -OK-, 201 -CREATED-, etc)

# API REST

```
GET /users HTTP/1.1
Host: the-website.com
Accept: application/json
```

application/xml  
was popular before  
JSON.

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 66
```

```
[
  {"id": 1, "name": "Alice"},
  {"id": 2, "name": "Bob"}
]
```

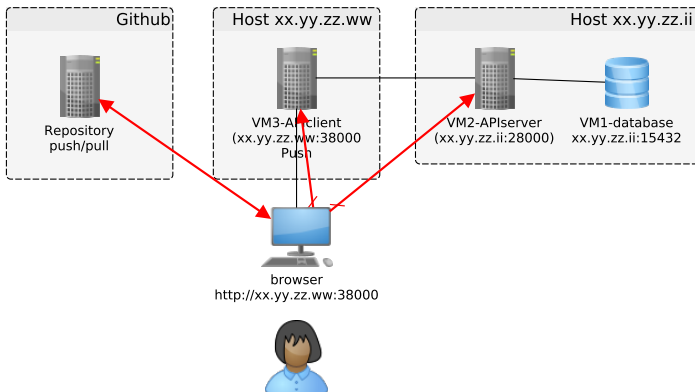
```
POST /users HTTP/1.1
Host: the-website.com
Accept: application/json
Content-Type: application/xml
Content-Length: 49
```

```
<user>
  <name>Claire</name>
</user>
```

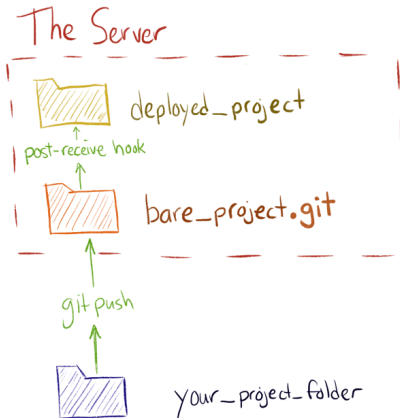
```
HTTP/1.1 201 Created
Location: /users/3
Content-Type: application/json
Content-Length: 28
```

```
{"id": 3, "name": "Claire"}
```

# Publicar código en las máquinas virtuales



# “push” no actualiza el código



# Git “hooks”

Edita:

```
# VM2
```

```
nano /home/si2/repo/p1base.git/hooks/post-receive
```

Agrega las siguientes líneas al archivo:

```
#!/bin/bash
```

```
GIT_WORK_TREE=/home/si2/repo/p1base git checkout -f
```